

```

import express from 'express';
import { boot, contextFor } from '../platform.js';
import * as db from '../db.js';
import { listPackages } from '../kernel/registry.js';
import { install, uninstall } from '../kernel/installer.js';
import { request, decide } from '../kernel/executor.js';
import { listCapabilities } from '../kernel/policy.js';
import { publicProfile } from '../kernel/projection.js';
import { verifyChain } from '../kernel/ledger.js';
import { drift } from '../kernel/canonical.js';
import { drain } from '../kernel/events.js';
import { provision, installOnDevice, removeFromDevice, openSession, closeSession,
import { fanOut } from '../kernel/fanout.js';
import { renderLinks, renderEmbed } from '../kernel/links.js';
import { publish, installAutomation, marketplace, watch } from '../kernel/automat
import { publicSurface, privateSurface, readResource, issueToken, revokeToken, au
import { listSlots, bind, bindings } from '../kernel/providers.js';
import { route, decisions } from '../kernel/router.js';
import { runHarness, runs } from '../kernel/harness.js';
import { propose, approve, reject } from '../kernel/builder.js';
import { importRepo, jobs, steps as importSteps } from '../kernel/importer.js';
import { verify as verifySignature } from '../kernel/signing.js';

const app = express();
app.use(express.json());

// Identity is stubbed here on purpose: swap this middleware for whatever
// session or token scheme you land on. Everything downstream only needs ctx.
async function withContext(req, res, next) {
  const personId = req.header('x-person-id') || null;
  const slug = req.params.slug || req.header('x-business') || null;
  if (!slug) return res.status(400).json({ error: 'business not specified' });
  try {
    req.ctx = await contextFor({ personId, businessSlug: slug });
    next();
  } catch (e) { res.status(404).json({ error: e.message }); }
}

// --- app store -----

app.get('/api/packages', (_req, res) => res.json(listPackages()));
app.get('/api/capabilities', (_req, res) => res.json(listCapabilities()));

app.get('/api/b/:slug/installs', withContext, async (req, res) => {

```

```

    res.json(await db.q(
      `select package_key, version, status, installed_at from install
        where business_id = $1 order by installed_at`, [req.ctx.businessId]));
  });

app.post('/api/b/:slug/installs', withContext, async (req, res) => {
  try {
    res.json(await install({
      businessId: req.ctx.businessId,
      packageKey: req.body.packageKey,
      settings: req.body.settings ?? {},
      grantTo: req.body.grantTo ?? ['owner'],
    }));
  } catch (e) { res.status(400).json({ error: e.message }); }
});

app.delete('/api/b/:slug/installs/:key', withContext, async (req, res) => {
  res.json(await uninstall({ businessId: req.ctx.businessId, packageKey: req.param
}));

// --- the one way to make anything happen -----

app.post('/api/b/:slug/invoke', withContext, async (req, res) => {
  const out = await request({
    ctx: req.ctx,
    capability: req.body.capability,
    input: req.body.input ?? {},
    resource: req.body.resource ?? '*',
    idempotencyKey: req.body.idempotencyKey ?? null,
  });
  await drain();
  res.status(out.error ? 400 : 200).json(out);
});

app.get('/api/b/:slug/approvals', withContext, async (req, res) => {
  res.json(await db.q(
    `select a.id, a.execution_id, a.asks_at, e.capability, e.input
      from approval a join execution e on e.id = a.execution_id
      where a.business_id = $1 and a.decided_at is null
      order by a.asks_at`, [req.ctx.businessId]));
});

app.post('/api/b/:slug/approvals/:executionId', withContext, async (req, res) => {
  const out = await decide({
    ctx: req.ctx, executionId: req.params.executionId,
    decision: req.body.decision, note: req.body.note ?? null,
  });
});

```

```

    await drain();
    res.json(out);
  });

// --- proof and drift -----

app.get('/api/b/:slug/receipts', withContext, async (req, res) => {
  res.json({
    chain: await verifyChain(req.ctx.businessId),
    receipts: await db.q(
      `select capability, verification, chain_hash, created_at from receipt
        where business_id = $1 order by created_at desc limit 100`, [req.ctx.busi
    });
  });

app.get('/api/b/:slug/drift/:key', withContext, async (req, res) => {
  res.json(await drift({ businessId: req.ctx.businessId, key: req.params.key }));
});

// --- public -----

app.get('/p/:slug', async (req, res) => {
  const profile = await publicProfile(req.params.slug);
  if (!profile) return res.status(404).json({ error: 'not found' });
  res.json(profile);
});

// --- workspace and device -----

app.get('/api/b/:slug/workspaces', withContext, async (req, res) => {
  res.json(await db.q(
    `select kind, state, region, persistent_volume, last_started from workspace
      where business_id = $1`, [req.ctx.businessId]));
  });

app.post('/api/b/:slug/workspaces', withContext, async (req, res) => {
  res.json(await provision({ businessId: req.ctx.businessId, region: req.body.reg
  });

app.get('/api/b/:slug/device/apps', withContext, async (req, res) => {
  res.json(await db.q(
    `select package_key, android_package, account_label, logged_in, last_seen
      from device_app where business_id = $1`, [req.ctx.businessId]));
  });

app.post('/api/b/:slug/device/apps', withContext, async (req, res) => {
  try {

```

```

        res.json(await installOnDevice({
            businessId: req.ctx.businessId,
            packageKey: req.body.packageKey,
            accountLabel: req.body.accountLabel,
            initialScreen: req.body.initialScreen ?? {},
        }));
    } catch (e) { res.status(400).json({ error: e.message }); }
});

app.delete('/api/b/:slug/device/apps/:key', withContext, async (req, res) => {
    res.json(await removeFromDevice({ businessId: req.ctx.businessId, packageKey: r
});

// The live screen the owner watches, and the same one automations drive.
app.post('/api/b/:slug/device/session', withContext, async (req, res) => {
    res.json(await openSession({ ctx: req.ctx, mode: req.body.mode ?? 'view' }));
});

app.delete('/api/b/:slug/device/session/:id', withContext, async (req, res) => {
    res.json(await closeSession({ businessId: req.ctx.businessId, sessionId: req.pa
});

app.get('/api/b/:slug/device/screen', withContext, async (req, res) => {
    res.json(await screenshot(req.ctx.businessId));
});

app.get('/api/b/:slug/sync', withContext, async (req, res) => {
    res.json(await db.q(
        `select c.provider_key, s.key, s.in_sync, s.last_checked, s.last_pushed
        from channel_sync_state s join connection c on c.id = s.connection_id
        where s.business_id = $1 order by c.provider_key`, [req.ctx.businessId]));
});

// Force a fan-out by hand. Normally nothing calls this: a canonical change does.
app.post('/api/b/:slug/sync/:key', withContext, async (req, res) => {
    res.json(await fanOut({ ctx: req.ctx, key: req.params.key }));
});

app.get('/api/b/:slug/repairs', withContext, async (req, res) => {
    res.json(await db.q(
        `select package_key, reason, step, screen, state, created_at from repair_item
        where business_id = $1 and state = 'open' order by created_at desc`, [req.c
});

// --- shared automations -----

app.get('/api/automations', async (_req, res) => res.json(await marketplace()));

```

```

app.post('/api/b/:slug/automations', withContext, async (req, res) => {
  try {
    res.json(await publish({ businessId: req.ctx.businessId, ...req.body }));
  } catch (e) { res.status(400).json({ error: e.message }); }
});

app.post('/api/b/:slug/automations/:key/install', withContext, async (req, res) =
  try {
    res.json(await installAutomation({ businessId: req.ctx.businessId, key: req.p
  } catch (e) { res.status(400).json({ error: e.message }); }
});

// --- the business's own MCP surface -----

app.get('/mcp/:slug', async (req, res) => {
  const s = await publicSurface(req.params.slug);
  if (!s) return res.status(404).json({ error: 'not found' });
  res.json(s);
});

app.get('/mcp/:slug/private', async (req, res) => {
  const token = (req.header('authorization') ?? '').replace(/^Bearer /i, '') || r
  const s = await privateSurface({ slug: req.params.slug, token });
  if (!s) return res.status(404).json({ error: 'not found' });
  if (s.error) return res.status(401).json(s);
  res.json(s);
});

app.post('/api/b/:slug/tokens', withContext, async (req, res) => {
  if (!req.ctx.membership) return res.status(403).json({ error: 'not a member' })
  const token = await issueToken({
    businessId: req.ctx.businessId, membershipId: req.ctx.membership.id,
    label: req.body.label ?? 'private surface',
  });
  res.json({ token, note: 'shown once' });
});

app.delete('/api/b/:slug/tokens/:id', withContext, async (req, res) => {
  res.json(await revokeToken({ businessId: req.ctx.businessId, tokenId: req.param
});

// --- the links page and embeds -----

app.get('/l/:slug', async (req, res) => {
  const html = await renderLinks(req.params.slug);
  if (!html) return res.status(404).send('not found');

```

```

    res.type('html').send(html);
  });

app.get('/embed/:slug/:section', async (req, res) => {
  const html = await renderEmbed(req.params.slug, req.params.section);
  if (!html) return res.status(404).send('not found');
  res.type('html').send(html);
});

// Public, unauthenticated, and still recorded: the search happens on the
// business's own page, so the business keeps it.
app.post('/public/:slug/availability/search', async (req, res) => {
  const business = await db.one(`select * from business where slug = $1`, [req.pa
  if (!business) return res.status(404).json({ error: 'not found' });
  const ctx = { businessId: business.id, business, person: null,
    membership: { id: null, role: 'system' }, system: true };
  const out = await request({ ctx, capability: 'availability.search',
    input: { ...req.body, surface: req.body.surface ?? 'links' } });
  res.json(out.result ?? { error: out.error });
});

app.get('/api/b/:slug/intent', withContext, async (req, res) => {
  res.json(await db.q(
    `select surface, requested_for, party_size, resource, matched, converted, cre
      from search_intent where business_id = $1 order by created_at desc limit 1
    [req.ctx.businessId]));
});

app.get('/mcp/:slug/:section', async (req, res) => {
  const r = await readResource(req.params.slug, req.params.section);
  if (!r) return res.status(404).json({ error: 'not found' });
  res.json(r);
});

// --- providers: every layer is a slot -----

app.get('/api/slots', (_req, res) => res.json(listSlots()));

app.get('/api/b/:slug/providers', withContext, async (req, res) => {
  res.json(await bindings(req.ctx.businessId));
});

app.post('/api/b/:slug/providers', withContext, async (req, res) => {
  try {
    res.json(await bind({
      businessId: req.ctx.businessId, slot: req.body.slot,
      packageKey: req.body.packageKey, config: req.body.config ?? {},

```

```

        priority: req.body.priority ?? 100,
      }));
    } catch (e) { res.status(400).json({ error: e.message }); }
  });

app.post('/api/b/:slug/think', withContext, async (req, res) => {
  try { res.json(await route({ ctx: req.ctx, task: req.body })); }
  catch (e) { res.status(400).json({ error: e.message }); }
});

app.get('/api/b/:slug/routing', withContext, async (req, res) => {
  res.json(await decisions(req.ctx.businessId));
});

// --- harness -----

app.post('/api/b/:slug/harness', withContext, async (req, res) => {
  try {
    res.json(await runHarness({ ctx: req.ctx, goal: req.body.goal, harnessKey: re
  } catch (e) { res.status(400).json({ error: e.message }); }
});

app.get('/api/b/:slug/harness', withContext, async (req, res) => {
  res.json(await runs(req.ctx.businessId));
});

// --- builder -----

app.post('/api/b/:slug/build', withContext, async (req, res) => {
  try { res.json(await propose({ ctx: req.ctx, intent: req.body.intent })); }
  catch (e) { res.status(400).json({ error: e.message }); }
});

app.post('/api/b/:slug/build/:planId/approve', withContext, async (req, res) => {
  try { res.json(await approve({ ctx: req.ctx, planId: req.params.planId })); }
  catch (e) { res.status(400).json({ error: e.message }); }
});

app.post('/api/b/:slug/build/:planId/reject', withContext, async (req, res) => {
  res.json(await reject({ ctx: req.ctx, planId: req.params.planId }));
});

// --- import -----

app.get('/api/import/steps', (_req, res) => res.json(importSteps));

app.post('/api/b/:slug/import', withContext, async (req, res) => {

```

```
    try {
      res.json(await importRepo({ ctx: req.ctx, source: req.body.source ?? 'github'
    } catch (e) { res.status(400).json({ error: e.message }); }
  });

app.get('/api/b/:slug/import', withContext, async (req, res) => {
  res.json(await jobs(req.ctx.businessId));
});

app.get('/api/packages/:key/:version/signature', async (req, res) => {
  res.json(await verifySignature({ packageKey: req.params.key, version: req.param
});

const port = process.env.PORT || 3000;
await boot();
for (const t of ['forms.submitted', 'execution.succeeded', 'canonical.fact_change
app.listen(port, () => console.log(`platform listening on :${port}`));
```